

LABORATORY MANUAL

SC2103: Digital System Design

Experiment 4:

Pipelining and Datapaths

(uP16 Processor)

**COLLEGE OF COMPUTING AND DATASCIENCE
NANYANG TECHNOLOGICAL UNIVERSITY**

SC2103 Laboratory 4

Pipelining and Datapaths (uP16 Processor)

Aim: To understand and verify the operation of a 5-stage pipelined processor (uP16), analyse its timing behaviour through behavioural and post-synthesis timing simulation, and implement the processor on the Basys 3 FPGA board.

Objectives:

Task	Objective Description
Task 1: Behavioural Simulation	To examine the uP16 pipelined processor structure, modify instruction memory contents, synthesize the design, and verify correct operation using behavioural simulation.
Task 2: Post-Synthesis Timing Simulation	To perform timing simulation and compare it with behavioural simulation to observe propagation delay effects.
Task 3: FPGA Implementation	To implement the uP16 processor on the Basys 3 board and verify correct program execution via 7-segment display and LEDs.

The uP16 is a 16-bit processor, and in Lab 4, a basic 5-stage pipelined implementation without data forwarding is introduced. It supports only full-word memory accesses (byte-level accesses are not supported). The Program Counter (PC) is 16 bits wide and holds the address of the next instruction to be fetched from memory. After each instruction is executed, the PC increments by 1. When the processor is reset, the PC is initialised to 0. The uP16 processor does not include any interrupt handling mechanism. A block diagram of the uP16 architecture is provided on the final page of the lab manual as Figure 1.

uP16 Register File: The uP16 processor contains eight general-purpose registers, labelled R_0 to R_7 .

- R_0 is permanently hardwired to the value 0. (Reading R_0 always returns 0, and writing to R_0 has no effect.
- R_1 to R_7 function as normal general-purpose registers and can be used to store data and intermediate results during program execution.
- R_6 may be used as a Stack Pointer (SP).
- R_7 may be used as a Return Address (RA) register during function calls.

uP16 Memory: The memory architecture of the uP16 processor consists of two distinct memory arrays:

1. Instruction Memory (IM): Configured as a 1024×18 -bit single-port ROM, and it stores all program instructions
2. Data/Stack Memory (DM): Configured as a 1024×16 -bit single-port RAM, and it is used for data storage and stack operations

Both memory blocks are implemented using explicitly instantiated Block RAM (BRAM) macros, rather than the IP Core Generator Wizard utilised in earlier laboratories.

This implementation approach is adopted to facilitate easier modification and re-initialisation of the Instruction ROM contents, as well as to improve visibility of memory contents during simulation. The direct instantiation of BRAM macros therefore provides greater flexibility and transparency in processor-level design and verification.

uP16 Instructions: All uP16 instructions are 18-bit words stored in the Instruction Memory (IM). The processor uses a single instruction format, referred to as the R-format, as shown in Table 1.

The instruction set is intentionally minimal and comprises a basic set of arithmetic, logical, memory access, and branch instructions, as listed in Table 2. Note that the architecture does not support explicit jump instructions. In addition, all arithmetic operations are performed using signed arithmetic.

Within the provided *uP16_Assembler.xlsx* file, each instruction corresponds to the five hexadecimal digits shown in the *Code* field. These five hexadecimal digits represent the complete 18-bit instruction word that is stored in the Instruction Memory. Open the *uP16_Assembler.xlsx* file from the zip file and check the same.

Table 1: The 18-bit instruction formats for the uP16 processor

Field	4 bits	3 bits	3 bits	8 bits
R-format	OpCode	Rd	Rs	Function/Immed

Table 2: The instruction set for the uP16 processor

Name	op	Funct/ Immed	Instruction	Effect(s)
nop	0x00	0x00	nop	Do nothing (instruction word = 0)
add	0x00	0x01	add <i>rd, rs</i>	RF[rd] = RF[rd] + RF[rs];
sub	0x00	0x02	sub <i>rd, rs</i>	RF[rd] = RF[rd] - RF[rs];
and	0x00	0x03	and <i>rd, rs</i>	RF[rd] = RF[rd] and RF[rs];
or	0x00	0x04	or <i>rd, rs</i>	RF[rd] = RF[rd] or RF[rs];
nand	0x00	0x05	nand <i>rd, rs</i>	RF[rd] = RF[rd] nand RF[rs];
nor	0x00	0x06	nor <i>rd, rs</i>	RF[rd] = RF[rd] nor RF[rs];
xor	0x00	0x07	xor <i>rd, rs</i>	RF[rd] = RF[rd] xor RF[rs];
not	0x00	0x08	not <i>rd, rs</i>	RF[rd] = not RF[rs];
mov	0x00	0x0A	mov <i>rd, rs</i>	RF[rd] = RF[rs];
lw	0x01	--	lw <i>rd, rs, immed</i>	RF[rd] <- memory(RF[rs]+immed);
sw	0x02	--	sw <i>rd, rs, immed</i>	RF[rd] -> memory(RF[rs]+immed);
lli	0x03	immed	lli <i>rd, rs, immed</i>	RF[rd] = sign_ext(immed); (<i>rs</i> is unused)
lui	0x04	immed	lui <i>rd, rs, immed</i>	RF[rd] = {immed, RF[rs][7:0]}
addi	0x05	immed	addi <i>rd, rs, immed</i>	RF[rd] = RF[rs] + sign_ext(immed);
	0x6-8			Reserved
beq	0x08	immed	beq <i>rd, rs, immed</i>	if (RF[rd] == RF[rs]) pc += sign_ext(immed) + 1;
bne	0x09	immed	bne <i>rd, rs, immed</i>	if (RF[rd] != RF[rs]) pc += sign_ext(immed) + 1;
blt	0x0A	immed	bltz <i>rd, rs, immed</i>	if (RF[rd] < RF[rs]) pc += sign_ext(immed) + 1;
bgt	0x0B	immed	bgtz <i>rd, rs, immed</i>	if (RF[rd] > RF[rs]) pc += sign_ext(immed) + 1;
	0xC-F			Reserved

Pre-Lab

Examine the Verilog source files *IF_stage.v*, *ID_stage.v*, *EX_stage.v*, and *Mem_stage.v*. As illustrated in Figure 1, the uP16 processor does not include a separate Write-Back (WB) stage; instead, register write-back occurs on the clock edge at the end of the Memory (Mem) stage.

- Within *ID_stage.v*, carefully review the implementation of the control module, with particular attention to the instruction decoding logic and the generation of control signals.
- Within *ID_stage.v*, carefully review the implementation of the regfile module, with particular attention to the read and write operations (read is combinational and write is sequential).
- Additionally, analyse the implementation of the 16-bit sign extension unit, which is defined as a single-line statement immediately below the module instantiations.

Task 1

Open a new project called Lab4, targeting the Artix-7 xc7a35tcpg236-1 FPGA. Extract the Verilog source files from *uP_src.zip* (inside *SC2103_Lab4.zip*) to a directory (called *up16_src*) inside the project directory. Add all the source files inside *up16_src* (as design sources) to the project. DO NOT add any of the files in *SC2103_Lab4.zip* other than the files inside *up16_src* at this point.

1. Open *Instruct_mem.v* (note that it is a sub module under *IF_stage*) and examine the *I_mem* module. Note that it has a port list with only clock, address, and data out (the minimum signal requirements for a synchronous single-port ROM). Also note the instantiated **BRAM_SINGLE_MACRO** macro with instance name **BRAM_SINGLE_MACRO_inst0** (below the .INIT initialisation information) and port identifiers DO to WE. Examine the signal mapping to the *I_mem* port list.
2. Examine the .INIT statements in the parameter section of the **RAMB16_S18** instance. This initialises the BRAM memory contents at startup (needed as it is a ROM).
 - There are 2 different initialisations. The .INIT_00 to .INIT_3F statements initialise the 1024x16 data section of the memory array, while .INITP_00 to .INITP_07 initialises the 1024x2-bit parity section. We are not using the parity bits as parity; instead, we are using them as extra memory bits to create 1024x18-bit memory. The important thing is that the two blocks need to be initialised separately. Note that everything is currently set to zero.
 - Next, open the uP16 assembler (*uP16_Assembler.xlsx*) and examine the blue shaded section. This allows you to enter instructions that are converted to assembly and binary, in the *Assembly Code* and *Code* columns, respectively. Replace the second *nop* (at row 4/memory address 0x02) with "**lli r3 ff**", like the instruction immediately above. Notice the change in the *Assembly Code* and *Code* columns. Examine the *RAM Data init* and *RAM Parity init* columns. They contain the .INIT and .INITP expressions. Replace the first three .INIT statements in *Instruct_mem.v* with the three non-zero .INIT statements from the *RAM Data init* column in *uP16_Assembler.xlsx*. Also, replace the first .INITP statement with the first .INITP statement from the *RAM Parity init* column in *uP16_Assembler.xlsx* (you may need to scroll down the page a bit to find the .INITP_00 statement). Save all files.
 - Now instructions are available inside the instruction memory
3. You can note that *uP16_top.v* will be the top module, and all the other Verilog files under the **up16_src** folder will be under the same. Add the constraints file (*uP16_top.xdc*) to the design.

4. Synthesise the design. Examine any errors or warnings. There are several warnings, but they should all be ignored. Open the Synthesised Design and check that **All user specified timing constraints are met**. This design has a 10ns clock (specified at the top of the *uP16_top.xdc* file). Note (but there is no need to do this step) that a constraint with a 7.5ns clock will also work, but clock periods less than that will fail. This means we could run this design up to about 133 MHz.
5. Add the testbench (*up16_top_tb.v*) as a simulation source to your design. Open the testbench. Note that it just has a reset (*Rst*) and a 10ns clock (*Clk*). Note that no other stimuli are needed, as the instructions are pre-initialised in the instruction memory (above).
6. Simulate the design. **Check that the default simulation radix is hexadecimal** (Select the settings icon at the above-right of the black waveform window). Zoom out till you can just see the digits in *IF_inst_d*. Observe that after *Rst* is de-asserted, the program counter (PC), *IF_currPC*, increments from 0000 each clock cycle. Also note that the processor instruction, *IF_inst_d*, which is read from the instruction memory (addressed by the PC), is one clock cycle later, as it is actually from the IF/ID pipeline register (see figure 1).
7. Add the register file (RF), *RegFile [0:7,15:0]*, and Data memory Write_Enab, Enable, Add_In, Data_in, Data_out simulation objects to the simulation. To do this, open the *uP16_top_tb* instance in the "Scope" pane. Open *uut*, and then the *T3* instance. Select the *EX_1* sub-instance (the various instances match the module instantiation instances in *uP_top.v*) until you have the *RegFile* element. Then drag the simulation object (from the "Objects" pane) to the simulation "Name" pane (Figure 2). Repeat for the *T7* instance, selecting the *DM_1* sub-instance, and then drag the required 5 instances. In the waveform window (in the "Name" pane), expand *RegFile*. **Save the simulation window to the Lab4 directory and add it to the project.**

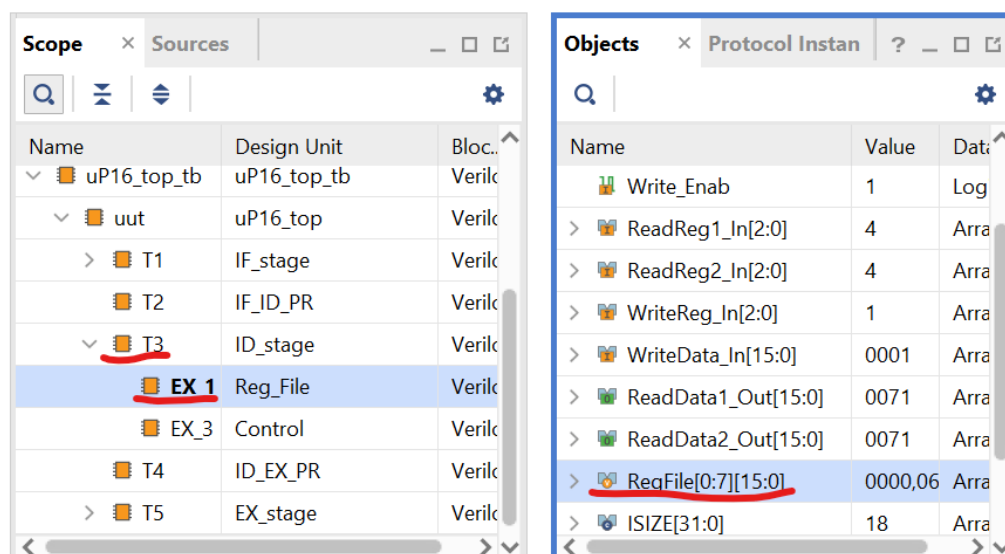


Figure 2: Simulation objects to the simulation

8. Set the run time for 1us and the Restart and Re-run the simulation for the time specified (you may need to do this twice if there is missing data). Maximise this window and zoom as necessary.
9. Click on the waveform at the second rising edge of *Clk* AFTER *Rst* goes low (at 135ns). The yellow cursor should move there. Observe that the first *Ili* instruction (*Ili r4, #71*) in the Excel file occurs at PC=0x0001. This instruction should put 0x71 into *RegFile*[4]. Note that it is written into the RF at the start of the 5th cycle after PC=0x0001. The next instruction is the one you added (*Ili r3 ff*). Note that *RegFile*[3] changes one cycle later but has a value of 0xFFFF. This is because the *Ili* sign extends the 8-bit value.

10. Next, scroll (and click) at 245ns. PC=0x000F is the first sw instruction (sw r4, r1, #0), which stores the contents of RF4 (0xF971) into the memory location pointed to by the sum of RF1 (0x0001) and the offset (0x00). Observe that the two memory enable, address, and data signals have the expected values and are active 2 cycles later. A two-cycle delay (rather than the expected 3 cycles) is correct, as Figure 1 shows that these signals are generated in the EX stage and bypass the EX_Mem pipeline register. But the memory is synchronous (clocked), and the write occurs on the next clock cycle (as indicated by the data on the Data_out signal appearing one clock cycle later).
11. Next, scroll (and click) at 275ns. PC=0x0012 is the first lw instruction (lw r6, r1, #0), which stores the contents of the previously written memory location to RF6. Observe that the memory enable and address are as expected and active 2 cycles later (Data_in is not active and contains only random data). The Data_out signal has valid data at the start of the 4th cycle after PC=0x0012, while RF6 is valid at the start of the 5th cycle after PC=0x0012, as expected.
12. Lastly, scroll across to PC=0x0015 (at 305ns). If you examine the Excel file, you will see that there are two consecutive add instructions (add r3, r1 followed by add r5, r3). In the simulation window, note that RF3=0x0002 and RF1=0x0001, so the sum should result in 0x0003 being written to RF3 at the start of the 5th cycle after the cursor. Observe that this occurs as expected. However, the 2nd add (which occurs 1 cycle later) should add RF5=0xFFFF (or -1) and RF3 (which should be 3 from the previous add), giving a result of 2. But note that RF5 (which does change 5 cycles later) is actually 0x0001. This is because data forwarding has NOT been implemented in this version of the uP16 processor. The value of RF3, which is used by the ALU, is the old stale value (0x0002) because the new value (0x0003) has not yet been written back to the register file. You can prove that by adding ALUData1 and ALUData2 from the ALU module into the simulation and seeing that the operands for the second add are retrieved before the writeback of 0x0003 to RF3 at 345ns. Hence, the result is -1+2 = 1, which is seen after 355ns. Note that there are earlier errors due to a lack of data forwarding. For example, look at PC=12 and PC=13. **Save the simulation window.**

Task 2

If not already done. Expand your Vivado window to full screen, then maximise your simulation window. Scroll the simulation window so that it starts just before the reset goes low. You should see from 115ns to somewhere near 300ns (or more). Use a snipping tool to capture the waveform window from 115ns to 300ns. Save this for later. **Now close the Simulation window** (by clicking on the X on the RHS of the blue SIMULATION bar).

Now run a post-synthesis timing simulation (note that if your synthesis is out of date, you may be asked to re-synthesise). To do this, select "Run Simulation" and then select "Post-Synthesis Timing Simulation".

1. As before, add the register file (RF) to your simulation. Select uut -> T3 -> EX_1, then in the objects window find \RegFile_reg[0] [15:0] (it should have a value of 0000 and is an array data type). Drag this to the simulation window. Then find \RegFile_reg[1] [15:0] through \RegFile_reg[7] [15:0] and drag each of them to the simulation window. Make sure they all have an array data type; otherwise, you probably have the wrong signals. Then maximise the waveform window. Ensure the run time is set to 1us (running for 100us takes too long). Restart and Re-run (till 1us). You may need to do this twice.
2. Click on 135ns (the yellow cursor should move there). This is the clock cycle at the start of the lli instruction (lli r4, #71) you examined in Task 1. Note that the point where PC=0x0001 now occurs 2/3rds of a clock cycle later. Now click at 175ns (the start of the 5th cycle) and observe that 0x0071 is written back into the register file during the 5th cycle (about 1/3rd of a clock cycle later, no longer at the start).
3. Also, observe that the memory is operating correctly. Remember, we used a sw

instruction to store the value in *RegFile*[4] to memory and then subsequently used a lw instruction to write back to *RegFile*[6]. These instructions occurred at PC=0x000F (245ns) and PC=0x0012 (275ns), respectively. Verify that the contents of *RegFile*[4] at 245ns are written back to *RegFile*[6] at about 320ns.

4. Compare the timing simulation with the saved behavioural simulation. Is there any difference (apart from the delay)? The values should be the same, but instead of changing at the rising edge of the clock, the signals change after the clock (about 2/3rd of a clock cycle later for the output signals and about 1/3rd of a clock cycle later for the register file values). This is because this simulation accounts for propagation delays through the circuit.
5. You can now close the simulation window. Also, close the Synthesised Design window.

Task 3

Now implement the uP16 design on the Basys3 board. The Basys3 board has been configured to display the lower 8 bits of the PC on the seven-segment display. The system clock has also been slowed down so that you can see the sequence of instructions as they are being executed. Additionally, the uP16 status is output to the board LEDs.

1. Remove the *uP16-top.xdc* constraints file and add the *uP16_BASYS3_top.xdc* constraints file.
2. Add the three design files, *uP16_BASYS3_top.v*, *seven_seg.v* and *clkgen.v*, as design sources.
3. At this point, again modify *seven_seg.v* to reflect your bench number.
4. Synthesise the design. Verify that any warnings can be ignored.
5. Implement the design and then generate the bitstream. Then turn on the power to the Basys3 FPGA board, open the Hardware Manager, and program the device
6. Verify that the 7-segment display matches the PC sequence from the saved behavioural simulation. It is: 00 to 0B, jump to 0F, continue sequentially to 1B, jump to 21, 22, 23, then jump back to 01 and repeat.

Inform your lab supervisor once you reach this point.

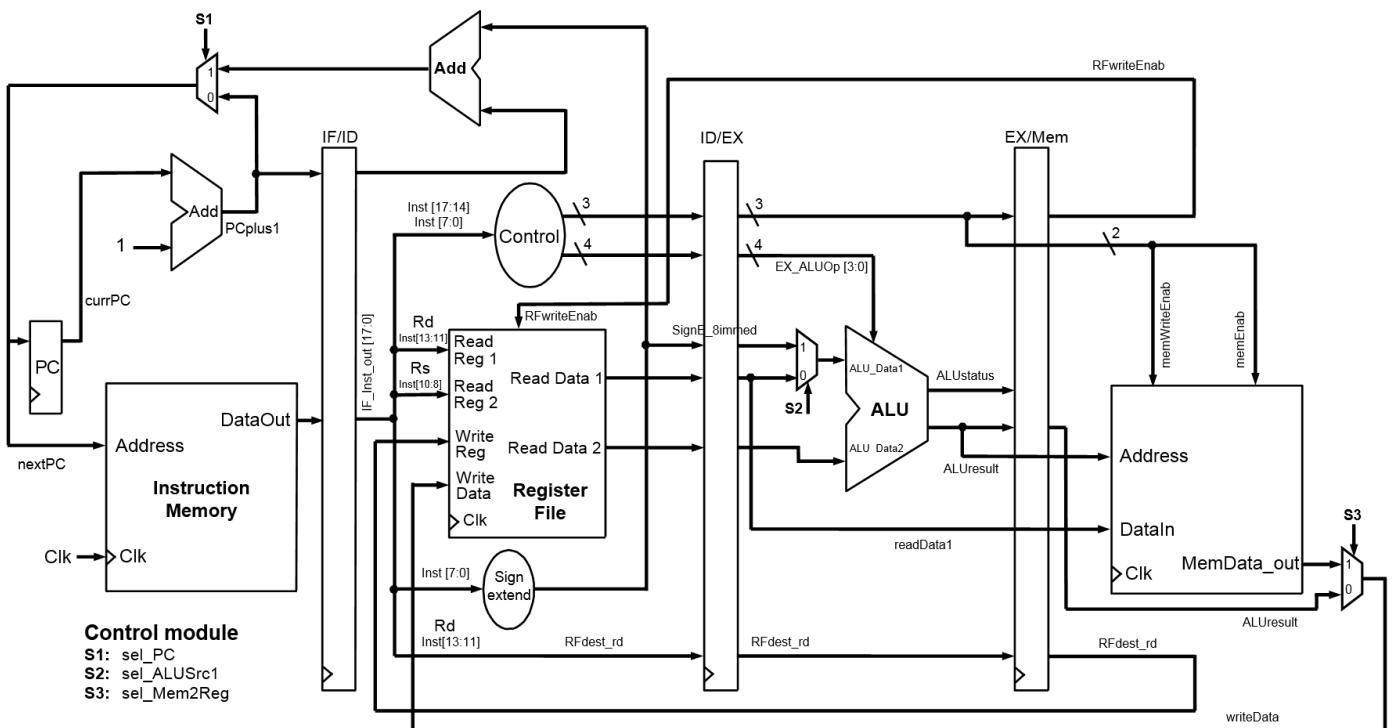


Figure 1: uP16 Block Diagram

Synthesiser warnings:

The following types of synthesiser warnings can

be ignored Task 1

[Synth 8-2507] parameter declaration becomes local in ??? with formal parameter declaration list

[Synth 8-689] width (32) of port connection 'DOADO' does not match port width (16) of module

'RAMB18E1' [Synth 8-3331] design Mem_stage has unconnected port ???

[Constraints 18-5210] No constraints selected for write.

Additional warning for Task 3

[Synth 8-3917] design uP16_BASYS3_top has port anode_L[3] driven by constant 1